Agent Evaluation Dashboard

Project 3 report: purpose, method, code walk-through, real findings and interview preparation

Repository folder: **03-eval-dashboard** | standard library only | 11 tests | output: one self-contained HTML page

The project in four sentences

It reads the run records and result files that Projects 1 and 2 already write (155 runs, 3 result files) and compares agent versions. Every rate gets a 95% confidence interval, and versions are compared on the **same questions**, so the page says *better*, *worse* or *within noise* instead of just showing two numbers. It also has a CI gate that fails when a new version is worse than the old one. The idea comes from a real mistake risk in Project 1: a score of 0.84 against 0.92 looks like a clear loss, but is it?

What it found on the real data

Project 1 comparison (38 questions)	Change	Fixed / broke	p (sign test)	Verdict
v1 pipeline to v2 agent	-7.9 pts	2 / 5	0.45	within noise
v2 to v2.1	+2.6 pts	1 / 0	1.00	within noise
v1 pipeline to v2.1	-5.3 pts	2 / 4	0.69	within noise

- **No Project 1 version difference is statistically supported.** Scores were v1 92% (35/38), v2 84% (32/38), v2.1 87% (33/38), with wide intervals (about 70% to 97%).
- **Speed is a clear difference:** median latency is 6.9 s for the pipeline and 43 s for the v2 agent (55 s for v2.1): a difference of the size the run records do show reliably.
- **Project 2:** 11/12 tasks passed (95% interval 65% to 99%) and 8/8 attacks resisted (68% to 100%). The wide intervals are the honest reading of 12 and 8 items.
- **'Within noise' means 'not shown', not 'equal'.** With 38 questions only large changes can be detected.

1. Purpose and the problem it solves

Problem	How the project answers it
Agent scores are usually one number from one run, and small differences get read as progress	Wilson intervals on every rate; the bar chart draws the interval as a whisker.
Two versions are tested on different questions or averaged away	Paired comparison on shared question ids: only the questions where the versions disagree carry information.
A regression slips into the code unnoticed	`evaldash gate` returns exit code 1 if the candidate drops more than the allowed amount or is significantly worse; usable in CI.
Dashboards often hide what they could not read	Skipped or unrecognised files are counted and listed on the page.
Speed and cost are ignored when quality is the headline	Run records feed latency percentiles, tokens, calls, status rates and failure reasons next to quality.

2. Architecture

```
<project>/data/runs/<version>/*.json   -> loader.load_runs      -> Run objects
<project>/evals/results/*.json        -> loader.parse_results  -> Suite (version -> items pass/fail)
                                              |
                          metrics.py  (rates, percentiles, compare versions)   <- stats.py (Wilson, bootstrap,
```

```
                                  |
report.py -> dashboard/index.html     cli.py -> summary | report | gate
```

Design decisions. Standard library only, so it installs anywhere and the maths is readable. Charts are inline SVG generated in Python, so the HTML has no JavaScript or CDN and can be opened offline or hosted as a static file. The input format is the shared run record that Projects 1 and 2 were built to write (schema_version 1), so the dashboard needed no changes to either project.

3. Code walk-through

File	Role	Key detail
stats.py	The maths	wilson(k,n); bootstrap_ci with a fixed seed (reproducible); exact sign_test_p; paired() returns delta, interval, fixed/broke counts, p and a verdict.
loader.py	Read the projects' files	Skips and reports invalid JSON, wrong schema_version and unrecognised result files. Three result shapes: several versions, a single task suite, an attack suite (success = resisted).
metrics.py	Numbers for the page	Per version: status rates, latency p50/p95 (zero latency from failed calls is not a measurement), mean calls/tokens, reasons. Per suite: overall and per group rates; comparisons of each version with the previous one and the first.
report.py	HTML renderer	Dark/light via CSS variables, phone-friendly tables, SVG bars with whiskers, 'how to read' note and 'limits' section on the page itself.
cli.py, __main__.py	Commands	report, summary, gate (exit 0 pass, 1 regression, 2 bad input).
tests/test_evaldash.py	11 tests	Known Wilson values, exact sign test, deterministic bootstrap, noise-vs-real cases, loader warnings, report content (no script tags), CLI exit codes.

4. The statistics, explained simply

Wilson interval (for 'passed 11 of 12')

The usual formula p +/- 1.96*sqrt(p(1-p)/n) breaks near 0% and 100% and for small n (it can give intervals above 100%). The Wilson interval does not. For 11/12 it gives 65% to 99%: the score is 92%, but with 12 questions the truth could plausibly be anywhere in that range.

Paired comparison and the sign test

If both versions answered the same 38 questions, questions both got right or both got wrong say nothing about which is better. Only the questions where they **disagree** matter. If v2.1 fixed 1 question and broke 0, that is weak evidence. If a version fixed 12 and broke 0, that is strong. The exact sign test asks: if the versions were equal, how likely is a split this lopsided? Below 5% I call it better or worse; otherwise 'within noise'. Example from the data: v1 to v2 fixed 2 and broke 5, p = 0.45, so the 8-point drop is not shown to be real.

Bootstrap interval

For the change in score, the questions are resampled with replacement 2,000 times, and the middle 95% of the changes is reported. The seed is fixed, so the same data always gives the same page.

Why a test with 'too little power' is still honest

A sign test with few disagreeing items rarely reaches significance. That is the right behaviour: it refuses to declare a winner it cannot support. The remedy is more questions or repeated runs, not a looser test.

5. How to run it

```
python -m venv .venv && .venv\Scripts\activate
pip install -e ".[dev]"
evaldash report --root ..          # writes dashboard/index.html from the sibling project folders
```

```
evaldash summary --root ..          # the same numbers as text
evaldash gate --root .. --project 01-knowledge-agent --suite agents --baseline v1 --candidate v2.1 --max-drop 0.
pytest                              # 11 tests
```

Gate in CI: run an evaluation, then call gate against the previous release's results. A failure exits non-zero and stops the merge. **Hosting:** the output is a static file; GitHub Pages is enough. Not deployed yet.

6. Alternatives and design choices

Choice	What I did	Alternative and trade-off
Presentation	Static HTML with inline SVG	Streamlit or Grafana: live and interactive, but need a server and dependencies. A static page can be hosted anywhere and is reproducible.
Intervals	Wilson and bootstrap in plain Python	scipy/statsmodels: fewer lines, extra dependency; writing them showed I understand them.
Comparison test	Exact sign test on disagreements	McNemar's test (nearly the same idea); a t-test on scores (wrong for pass/fail data); Bayesian comparison (richer, harder to explain).
Tracking tools	My own run-record format	LangSmith, Langfuse, W&B;, MLflow: far richer and production-grade; this project shows the underlying method and has no vendor lock-in.
Quality metric	Whatever each project's scorer says	LLM-as-judge scores could be added, but are noisy and add their own error.
Multiple comparisons	Reported as is, no correction	Bonferroni or similar if many versions are compared; here there are 3 comparisons and none is significant anyway.

7. Honest limits

- Success definitions differ per suite, so numbers are not comparable across suites.
- Test sets are small (8 to 38 items). Real conclusions need more questions and repeated runs.
- Run counts differ between versions in the run records (re-runs, partial runs; v2.2 has only 20 runs because of an API quota), so the run-record table is descriptive, not a controlled comparison.
- Cost shows 'not recorded' because the free tier did not report it.
- Static page: regenerate after each evaluation. No deployment yet. No CI workflow has been run.

8. Interview preparation

Q. What does this project do and why build it?

It compares agent versions with confidence intervals and paired tests, so I can say whether a change is real. I built it because in Project 1 a headline difference of several points turned out to be within noise.

Q. Why not just compare average scores?

On 38 questions a few flipped answers move the average by several points. Without an interval or a paired test you can't tell luck from improvement.

Q. Explain the sign test to a non-statistician.

Ignore questions both versions got the same. Of the rest, count how many each version won. If the versions were equal it would be like a coin toss; the test says how unlikely the split is for a fair coin.

Q. Your result says 'within noise' for everything in Project 1. Isn't that a useless result?

It is the correct result. It tells me my earlier numbers can't support a claim in either direction, which changes what I should do next: more questions and repeated runs, not tuning.

Q. What did the dashboard reveal that you didn't already know?

That latency differs by a large and reliable margin: median 43 s for the agent against 6.9 s for the pipeline, while quality differences are not demonstrated. For a product that changes the trade-off discussion.

Q. Why Wilson and not the normal approximation?

Wilson stays within 0 to 100% and is accurate for small samples and extreme rates; the normal formula fails exactly in the cases I have (11/12, 8/8).

Q. Why not use LangSmith or Langfuse?

They are better for production tracing. I wanted to show the evaluation method itself, with no vendor dependency; the run-record schema could be exported to them.

Q. How would you use it in a team?

Run the evaluation on every pull request, call gate against the main branch's results, and publish the HTML as a build artifact so reviewers see intervals, not single scores.

Q. What would you improve?

Repeat each evaluation several times, add per-question cost, track results over time, add a larger question set with held-out questions, and a multiple-comparison correction when many versions are compared.

Resume line (backed by what was measured above)

Built an evaluation dashboard (Python, standard library) that compares agent versions using Wilson intervals, paired sign tests and bootstrap intervals, with a CI regression gate; it showed that a 0.92 vs 0.84 gap across 38 questions was statistically indistinguishable from noise while latency differed 6x; 11 automated tests.